🚀 **Hands-On AWS WAF Lab**

📘 Lab 1: Block Specific IPs with AWS WAF + Test Using `curl`

🎯 Goal:

Block traffic from a specific IP using a Web ACL attached to **CloudFront** or **ALB**, and verify using `curl`.

🧪 Steps:

1. **Create Web ACL:**
    - Go to **AWS WAF > Web ACLs > Create**
    - Name: `ip-block-waf`
    - Region: `Global` (for CloudFront) or choose ALB region
    - Resource type: CloudFront or Application Load Balancer
    - Associate the ACL to your resource
2. **Create IP Set:**
    - Go to **IP Sets > Create**
    - Name: `blocked-ip-set`
    - Add your **public IP address** or simulated IP to block (`x.x.x.x/32`)
3. **Create Rule:**
    - Rule name: `BlockBadIP`
    - Rule type: IP match

- Use `blocked-ip-set`
- Action: **Block**
- Priority: 1

Test with curl:

curl -I https://your-distribution-or-alb-endpoint

4.
5. **Expected:** `403 Forbidden` response.

📘 Lab 2: Use AWS Managed Rule Set to Block Common Attacks

🎯 Goal:

Apply **AWSManagedRulesCommonRuleSet** to protect your app from OWASP Top 10 threats.

🧪 Steps:

1. Go to your Web ACL > Add rules > **Add managed rule groups**
2. Select:
   - Vendor: `AWS`
   - Rule group: `AWSManagedRulesCommonRuleSet`
3. Set action to `Count` initially (optional)
4. Save and deploy.

🧪 Test:

Use tools like:

curl -H "User-Agent: BadBot" https://your-app-url

📊 Check logs/CloudWatch metrics for detection.

📘 Lab 3: Attach AWS WAF to API Gateway & Inspect Logs

🎯 Goal:

Protect your API from unwanted requests and monitor logs.

🧪 Steps:

1. Create a **REST API** using Amazon API Gateway.
2. Deploy it (example: `/hello` endpoint).
3. Create a **Web ACL** and attach it to the API Gateway.
4. Add a simple rule:
    - Block requests from specific IP or path
5. Enable logging:
    - Go to Web ACL > **Logging and metrics**
    - Enable logging to **S3** or **CloudWatch Logs**

Access the endpoint:

curl https://api-id.execute-api.region.amazonaws.com/stage/hello

6.
7. Observe log records in CloudWatch or S3.

📘 Lab 4: Simulate SQL Injection and Observe WAF Response

🎯 Goal:

Use `curl` to simulate SQLi and let AWS WAF block it using managed rules.

🧪 Steps:

1. Add **AWSManagedRulesSQLiRuleSet** to your Web ACL
2. Action: **Block**

Deploy and test with:

curl "https://your-app.com/login?user=admin'--"

3.
4. Observe:
    - `403 Forbidden` response
    - Log entry indicating SQLi rule triggered

📘 Lab 5: Enable WAF Logging to Amazon S3 & Analyze

🎯 Goal:

Enable full request/response logging to an S3 bucket.

🧪 Steps:

1. Go to Web ACL > **Logging and metrics**
2. Choose **Enable logging**
3. Select or create an S3 bucket (e.g., `waf-log-bucket`)

4. Save

🔍 Analyze:

- Use Athena or CloudWatch Insights to query logs
- Log entries look like:

```
{
  "action": "BLOCK",
  "ruleGroupList": [...],
  "httpRequest": {
    "clientIp": "x.x.x.x",
    "uri": "/login",
    "headers": [...]
  }
}
```

🏁 Deliverables for Students

- ✅ Screenshot of Web ACLs and rules
- ✅ Test output (`curl` or browser) showing blocked access
- ✅ CloudWatch/S3 log entries with evidence of rules triggering
- ✅ Short reflection: What was blocked, why, and how WAF helped?

📦 Bonus: Common curl Commands for Testing WAF

```
# Normal request (should pass)
curl -I https://your-app

# SQL Injection test
curl "https://your-app.com/search?q=test' OR 1=1--"

# Cross-site scripting
```

```
curl "https://your-app.com/page?input=<script>alert(1)</script>"

# Bad User-Agent
curl -H "User-Agent: BadBot" https://your-app

# Send custom header
curl -H "X-Custom-Test: attack" https://your-app
```
